م 2

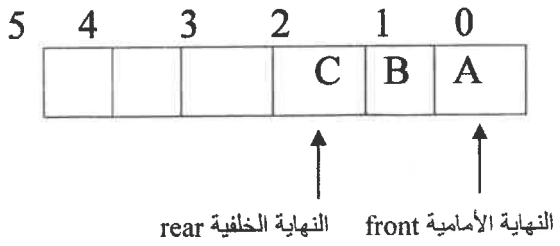
هياكل بيانات

م. بشير محمد

22.10.2025

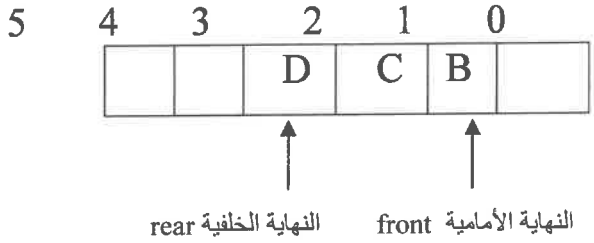
٣-٣ الطابور queue

هو هيكل تسلسلي (sequential) تكون فيه عمليات الاضافة في النهاية الخلفية rear وعمليات الحذف في النهاية الاخرى (الامامية) front كما في الشكل التالي:



حيث نلاحظ ان العنصر A في مقدمة الطابور يليه العنصر B ثم C وعند اضافة عنصر جديد يكون موقعه بعد C ، اما عند حذف عنصر من الطابور تكون عملية الحذف من النهاية

الأمامية اي حذف العنصر A ويصبح الشكل اعلاه بعد اضافة العنصر D وحذف العنصر A كالاتي:



نجد أن الطابور يفيد في الفعاليات التي تتضمن جدولة الأعمال حسب ترتيب وصولها او طلبها ويمكن تلخيص هذا بالعبارة الاتية: أول من يدخل اول من يخرج FIRST IN FIRST OUT (FI FO). أي ان من يصل اولا يحصل على الخدمة اولا وتسمى عملية الإضافة الى الطابور ENQueue او Insertion اما عملية الحذف فتسمى DEQueue او Deletion .

٣-٣-١ تمثيل الطابور باستخدام المصفوفة Array Representation of queue

يطبق الطابور باستخدام مصفوفة احادية بالسعة المطلوبة (size) وبالنوع المناسب لنوع البيانات (data type) التي ستخزن فيه (int , float , الخ) مع استخدام :

- المتغير (rear) كمؤشر يشير الى موقع العنصر الاخير في الطابور.
- المتغير (front) كمؤشر يشير الى موقع العنصر الاول في الطابور

ان قيمة المؤشرين في الحالة الابتدائية (rear=-1,front=0) عندما يكون طابور خاليا(empty) من العناصر .تنفذ عملية اضافة عنصر الى الطابور بعد تحديث قيمة المؤشر (rear) ليشير الى الموقع الجديد بعد موقع اخر عنصر ، اما عند تنفيذ عملية حذف عنصر من الطابور فيحدث المؤشر (front) ليشير الى موقع العنصر التالي بعد حذف العنصر في المقدمة،ولنفترض ان لدينا الطابورQ سعته ٦ عناصر وننفذ عليه سلسلة العمليات الاتية:

الحالة	المؤشر	المؤشر	q[0]	q[1]	q[2]	q[3]	q[4]	q[5]
	F	R						
- الطابور خالي	0	-1	-	-	-	-	-	
- اضافة العنصر A	0	0	A	-	-	-	-	-
- اضافة العنصر B	0	1	A	B	-	-	-	
- اضافة العنصر C	0	2	A	B	C	-	-	
- حذف عنصر		1		2	-	B	C	-
- اضافة العنصر D	1	3	-	B	C	D	-	
- اضافة العنصر E	1	4	-	B	C	D	E	
- حذف عنصر		2	4	-	-	C	D	E
- حذف عنصر		3		4	-	-	-	D	E

ويعرف الطابور برمجياً باستخدام العبارات البرمجية التالية:

عملية الاضافة للطابور ADD TO QUEUE :تعتمد الخطوات الاتية لاضافة عنصر واحد الى الطابور:

١- التحقق بان الطابور غير مملوء (NOT FULL) اي ان المؤشر (rear!=SIZE-1) لتجنب حالة الفيض (OVER FLOW) وتعذر تنفيذ عملية الاضافة عند ذلك .

٢- تحديث قيمة المؤشر (rear=rear+1) ليشير الى الموقع التالي.

٣- اضافة العنصر الجديد في الموقع الجديد QUEUE [rear]

عملية الحذف من الطابور DELETE FROM QUEUE :نعتمد الخطوات الاتية لحذف عنصر واحد من الطابور:

١- التحقق بان الطابور غير خال (NOT EMPTY) اي ان المؤشر (rear>=front) لتجنب حالة الغيض (UNDER FLOW) وتعذر تنفيذ عملية الحذف.

٢- اخذ العنصر من الموقع الذي يشير اليه المؤشر (front) وخزنه وقتيا في متغير مستقل وليكن ITEM=QUEUE[front]

٣- تحديث قيمة المؤشر (front=front+1) ليشير الى موقع العنصر الاتي للعنصر الذي تم حذفه.

ملاحظة: هنا ايضا يتضح ان الخطوتين٣(٢) في عملية الحذف معكوسة الترتيب عنها في عملية الاضافة.

٣-٣-٣ خوارزميات الطابور Queue's Algorithms

ادناه مجموعة من الخوارزميات لتغطية العمليات التي تنفذ على الطابور:

١- خوارزمية الاضافة Add Queue

```
If      queue is full
Then over flow ←      true
Else
      Over flow ←     false
      Rear ←     rear+1
       Queue (rear) ←      new element
```

٢-خوارزمية الحذف delete queue

```
If      queue is empty
Then under flow ←     true
Else
       Under flow ←      false
        Element ←          queue(front)
        Front ←        front +1
```

٣-خوارزمية ملئ الطابور full queue

هذه الخوارزمية للتحقق من الطابور ان كان مملوء ام لا اعتمادا على قيمة المؤشر(rear) قبل عمليات الاضافة.

```
If          rear= size-1
Then        fullqueue ←                true
```

Else fullqueue ← false

٤- خوارزمية خلو الطابور empty queue

هذه الخوارزمية للتحقق من الطابور ان كان خاليا ام لا اعتمادا على قيمة المؤشر (front) قبل عملية الحذف.

If rear<front
Then emptyqueue ← true
Else empty queue ← false

٥- خوارزمية إخلاء (تفريغ) الطابور Clear Queue

هذه الخوارزمية تستخدم لغرض تهيئة الطابور واخلائة من العناصر بجعل قيمة كل من المؤشرين (front =0 ,rear=-1)

Fornt ← 0
Rear ← -1

٣-٣-٣ البرامج الفرعية لتنفيذ عمليات الطابور

فيما يلي مجموعة من البرامج الفرعية (Functions , Procedures)أعدت بنفس أسلوب البرامج الفرعية للمكدس مع افتراض وجود التعريف التالي للمتغيرات في مقدمة البرنامج .

```
#include<iostream.h>
#include<conio.h>
#include<stdlib.h>
#define size 10
int q[size];
int item;
int front,rear;
```

١- برنامج فرعي لاخلاء الطابور

```
void clearqueue()
{
 rear=-1;
 front=0;
}
```

نلاحظ عدم الحاجة للمرور على جميع مواقع المصفوفة والاكتفاء فقط بجعل قيمة المؤشر rear مساويا 1- و المؤشر front مساويا 0 للصفر .(يستدعى هذا البرنامج الفرعي في البداية لجعل الطابور خاليا) .

٢- برنامج فرعي للتحقق من امتلاء الطابور

```
int fullqueue()
{
 if(rear>=size)
 return(1);
```

```
else return(0);
}
```

هذه الدالة (function) تقوم بارجاع قيمة(1) اي true عندما يكون الطابور مملوء وتقوم بارجاع قيمة(0) اي false عندما يكون الطابورغير مملوء. (يستدعى هذا البرنامج الفرعي داخل البرنامج الفرعي (procedure insert Queue)الذي ينفذ عملية الاضافة).

٣-برنامج فرعي للتحقق من خلو الطابور

```
int emptyqueue()
{
 if(rear<front)
 return(1);
 else return(0);
}
```

هذه الدالة (function) تقوم بارجاع قيمة (1) اي true عندما يكون الطابور فارغا وتقوم بارجاع قيمة (0) اي false عندما يكون الطابورغير فارغ. (يستدعي هذا البرنامج الفرعي داخل البرنامج الفرعي (Procedure Delete Queue)الذي ينفذ عملية الحذف) .

٤- برنامج فرعي لاضافة عنصر واحد الى الطابور

```
void addqueue(int item)
{
 rear++;
 if(fullqueue())
 {
  cout<<"error queue is full"<<endl;
  cout<<"press any key to exit"<<endl;
  getch();
  exit(0);
 }
 else
     q[rear]=item;
}
```

ملاحظة : يمكن استدعاء هذا البرنامج الفرعي داخل البرنامج الرئيسي (main Program)بأي عدد من المرات باستخدام أحد ايعازات التكرار مثل (for,do_while) بقدر عدد العناصر المطلوب أضافتها .

٥- برنامج فرعي لحذف عنصر واحد من الطابور

```
void deleteq()
{
 if(emptyqueue())
```

تمرين: اكتب برنامجا فرعيا لاضافة ٥ عناصر الى الطابور (LINE) الذي سعته ٢٥ عدد صحيح.

```
const size=25;
int line[size];
void addqueue()
{
 int i,item;
 for(i=0;i<5;++i)
  {
   if(rear>=size-1)
    {
     cout<<"error…queue is full"<<endl;
     cout<<"press any key to exit"<<endl;
     getch();
     exit(0);
     }
     else
      {
       rear++;
      cin>>item;
      line[rear]=item;
      }
     }
}
```

تمرين : اكتب برنامجا فرعيا لقراءة جملة string تتكون من جزأين (two substring) يفصلها الرمز ،.، ثم التحقق من كونهما متطابقتان ام لا باستخدام الطابور.

٣-٤ الطابور الدائري circular queue

لاحظنا انه عند اضافة عنصر الى الطابور يتطلب تدقيق (فحص) قيمة المؤشر (rear=size-1) اذ ان ذلك يعني ان الطابور مملوء حتى وان كانت هنالك مواقع خالية في مقدمة الطابور كما في الشكل الاتي:

Queue:

7	6	5	4	3	2	1	0
Y	X	N	M				

rear=7 front=4

أي أننا سنخسر مساحة خزنية دون استخدام ، ولتجنب هذه الحالة نستطيع استخدام الطابور استخداما دائريا وذلك بان نسمح للمؤشر (rear) بالدوران الى النهاية الامامية للطابور (wrap-around) عندما يكون هنالك مواقع خالية فيها.

ان خصائص هذا الطابور هي كاتي :

١- ان المؤشر front يشير الى الموقع الذي امام اول عنصر في الطابور.

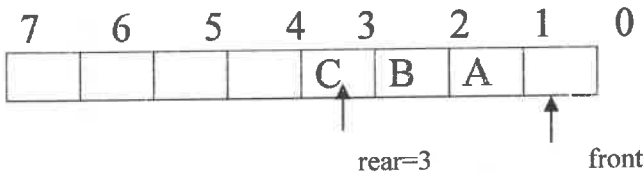
٢- أن المؤشر rear يشير الى موقع العنصر الاخير في الطابور.

٣- عندما يصل المؤشر rear الى الموقع الاخير في الطابور اي rear=size-1 نجعله يدور الى البداية اي rear=0 وكذلك الحال بالنسبة للمؤشر front.

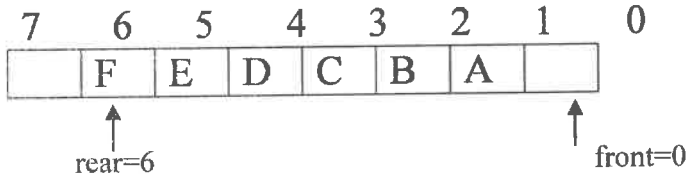
٤- ان اكبر عدد من العناصر التي يستوعبها هذا الطابور هو size-1 لان المؤشر front يشير الى الموقع الخالي امام اول عنصر في الطابور.

ولنأخذ هذه الأمثلة التوضيحية الآتية لطابور دائري سعته (8):

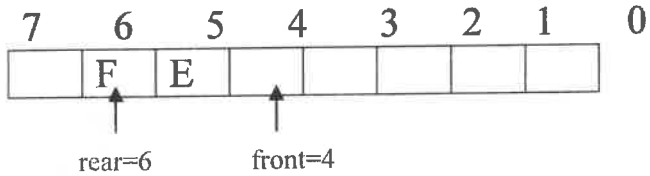
الحالة الأولى : يحتوي الطابور ثلاثة عناصر C,B,A



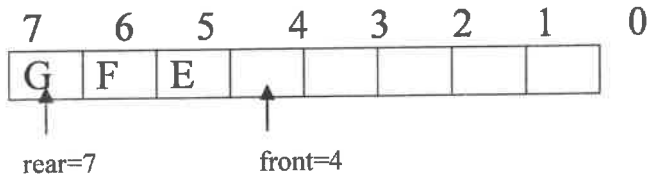
الحالة الثانية : بعد اضافة العناصر F,E,D



الحالة الثالثة : بعد حذف العناصر D,C,B,A

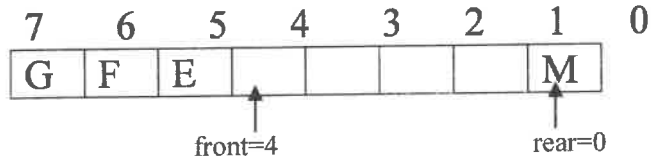


الحالة الرابعة: بعد اضافة العنصر G



لاحظ هنا في الطابور البسيط يعتبر الطابور مملوء لان rear=SIZE-1=7 مع انه يوجد مواقع خالية في المقدمة اما في الطابور الدائري فنستطيع اضافة عنصر كما في الحالة التالية.

الحالة الخامسة :لاضافة العنصر M لاحظ دوران المؤشر rear الى الجهة الامامية بعد وصوله الى الموقع الاخير في الطابور .



الحالة السادسة : بعد اضافة العناصر x,p,n لاحظ هنا ان الطابور اصبح مملوءا لاننا اذا اردنا اضافة عنصر فيجب تحديث قيمة المؤشر rear لصبح مساويا 4 ويكون مساويا لقيمة المؤشر front=4 الذي يجب ان يبقى موقعه خاليا كما ان سعة الطابور هي size-1 اي ان 7=1-8 وهذا ما يحتويه حاليا ، لذا فيتعذر الاضافة في هذه الحالة.

7	6	5	4	3	2	1	0
G	F	E		X	P	N	M
			front=4	rear=3			

برنامج فرعي لاضافة عنصر الى الطابور الدائري

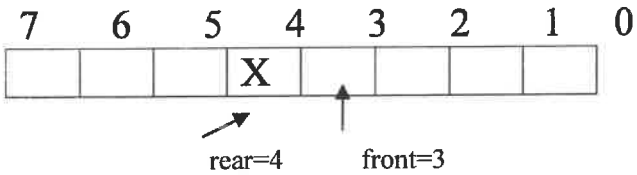
```
void addcq(int item)
{
 if(rear>=size-1)
 rear=0;
 else
 rear++;
 if(rear==front)
 {
  cout<<"error...circular queue is full"<<endl;
  cout<<"press any key to exit"<<endl;
  getch();
  exit(0);
 }
 else
cq[rear]=item;
}
```

برنامج فرعي لحذف عنصر من الطابور الدائري

```
void deletecq()
{
 if(rear==front)
 {
  cout<<"error...circular queue is empty"<<endl;
  cout<<"press any key to exit"<<endl;
  getch();
  exit(0);
 }
 else
 {
  if(front>=size-1)
```

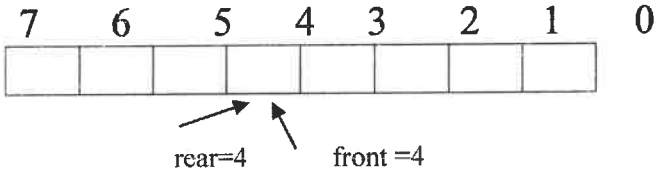
```
front=0;
else front++;
item=cq[front];
}
}
```

توضيح : لناخذ طابورا دائريا سعته ٨ عناصر يحتوي حاليا على عنصر واحد هو x كما في الرسم التوضيحي ادناه حيث المؤشر front=3 وقيمة المؤشر rear=4 .



عند حذف عنصر واحد ماذا سيحصل :

- نحدث قيمة المؤشر front:=front+1 اي يصبح front =4
- ناخذ العنصر من موقعه item =q [front]
- يصبح شكل الطابور الدائري كالاتي:



rear=front=4 والطابور خال من العناصر لذا فان اي عملية حذف لاحقة كما في البرنامج الفرعي deleteCQ يتعذر تنفيذها.

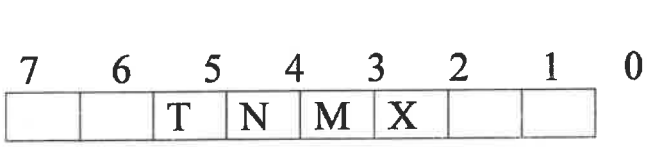
ملاحظة مهمة: نجد ان الشرط نفسهrear==front يستخدم في البرنامج الفرعي ADDCQ ليعني ان الطابور مملوء وفي البرنامج الفرعي DELETECQ ليعني ان الطابور خال وهذا لا يعني التناقض لان تسلسل الخطوات يختلف اذ يرد عند الاضافة بعد تحديث قيمة المؤشر rear ويرد عند الحذف في البداية.

٣-٥ الطابور الثنائي (المزدوج)(DOUBL ENDED QUEUE(DEQEUE)):

هو هيكل تسلسلي يمكن اضافة او حذف العنصر من اي من طرفية ويمثل بمصفوفة احادية مع اربعة مؤشرات هي:

F1 يشير الى موقع اول عنصر في الطابور عند استخدامه من الجهة اليمنى.
R1 يشير الى موقع اخر عنصر في الطابور عند استخدامه من الجهة اليمنى.
F2 يشير الى موقع اول عنصر في الطابور عند استخدامه من الجهة اليسرى.
R2 يشير الى موقع اخر عنصر في الطابور عند استخدامه من الجهة اليسرى.

كما في الشكل التالي



R1 F1

اذ يمكن حذف العنصر X باستخدام المؤشر F1 واضافة العنصر T باستخدام المؤشر R1 ، كما يمكن حذف العنصر T باستخدام المؤشر F2 واضافة عنصر بعد العنصر X باستخدام المؤشر R2 .

برنامج -٤ تمثيل الطابور QUEUE وعملياته

```
#include<iostream.h>
#include<conio.h>
#include<stdlib.h>
#define size 10
int q[size];
int item;
int front,rear,choice,i,l,m;
int fullqueue()
{
 if(rear>=size)
 return(1);
 else return(0);
}
int emptyqueue()
{
 if(rear<front)
 return(1);
 else return(0);
}
void addqueue(int item)
{
 rear++;
 if(fullqueue())
 {
  cout<<"error queue is full"<<endl;
  cout<<"press any key to exit"<<endl;
  getch();
  exit(0);
 }
 else
 q[rear]=item;
```

```
}
void deleteq()
{
 if(emptyqueue())
 {
  cout<<"error queue is empty"<<endl;
  cout<<"press any key to exit"<<endl;
  getch();
  exit(0);
 }
 else
 {
  item=q[front];
  front++;
 }
}
void main()
{
 clrscr();
 rear=-1;
 front=0;
 do
 {
  cout<<"representation of queue operation"<<endl;
  cout<<"--------------------------------"<<endl;
  cout<<"1-insertion operation      "<<endl;
  cout<<"2-deletion operation       "<<endl;
  cout<<"3-display the content of the queue"<<endl;
  cout<<"4-exit                     "<<endl;
  cout<<"select your choice"<<endl;
  cin>>choice;
  switch (choice)
   {
    case(1):
    {
       cout<<endl<<"how many elements you like to enter";
       cin>>m;
```

```
            for(i=0;i<m;i++)
            {
              cout<<"enter the new element"<<endl;
              cin>>item;
            addqueue(item);
              cout<<endl<<"rear="<<rear<<",front="<<front";
            }
            break;
        }
        case(2):
         {
            cout<<"how many elements you want to delete"<<endl;
            cin>>l;
            for(i=0;i<l;i++)
            deleteq();
            break;
          }
        case(3):
         {
         if(rear<front)
           cout<<"error queue is empty"<<endl;
           else
         {
            cout<<"the content of the queue is:"<<endl;
            for(i=rear;i>=front;i--)
            cout<<endl<<q[i]<<endl;
           }
          break;
         }
        }
      }while(choice!=4);
    }
```

برنامج -٥ تمثيل الطابور الدائري (CIRCULAR QUEUE) وعملياته

```
#include<iostream.h>
#include<conio.h>
#include<stdlib.h>
```

```
#define size 6
int cq[size];
int front,rear,choice,i,l,m;
int item;
void addcq(int item)
{
 if(rear>=size-1)
 rear=0;
 else
 rear++;
 if(rear==front)
 {
  cout<<"error...circular queue is full"<<endl;
  cout<<"press any key to exit"<<endl;
  getch();
  exit(0);
 }
 else
 cq[rear]=item;
}
void deletecq()
{
 if(rear==front)
 {
  cout<<"error...circular queue is empty"<<endl;
  cout<<"press any key to exit"<<endl;
  getch();
  exit(0);
 }
 else
 {
  front++;
  if(front>=size-1)
  front=0;
  else
  item=cq[front];
  cq[front]=0;
```

```
}
}
void main()
{
clrscr();
rear=0;
front=0;
for(i=0;i<size;i++)
cq[i]=0;
do
{
cout<<"representation of queue operation"<<endl;
cout<<"--------------------------------"<<endl;
cout<<"1-insertion operation      "<<endl;
cout<<"2-deletion operation       "<<endl;
cout<<"3-display the content of the circular queue"<<endl;
cout<<"4-exit                     "<<endl;
cout<<"select your choice"<<endl;
cin>>choice;
switch (choice)
  {
  case(1):
  {
     cout<<"how many elements you like to enter"<<endl;
     cin>>m;
     for(i=0;i<m;i++)
    {
      cout<<"enter the new element"<<endl;
      cin>>item;
      addcq(item);
       cout<<"rear="<<rear<<"  front="<<front;
     }
     break;
  }
     case(2):
    {
       cout<<"how many elements you want to delete"<<endl;
```

```
            cin>>l;
            for(i=0;i<l;i++)
            deletecq();
            break;
           }
          case(3):
          {
            cout<<"the content of the circular queue is:"<<endl;
            for(i=0;i<size;i++)
            cout<<endl<<cq[i]<<endl;
            cout<<"rear="<<rear<<"  front="<<front;
            break;
           }
          }
      }while(choice!=4);
    }
```

أسئلة الفصل

1- Let s be a stack of size 15 character, write a procedure push to add one element.

2- Write a procedure to pop three elements from the stack TABLE of size 20 integers.

3- Let s for stacking an element in the stack, and U for un stacking.
If the order of stack input stream is 1 2 3 4 5

a- what is the output if we execute the following operations SSUUSSSUUU

b- Which of the following permutations can be obtained as output stream (explain the reason for each case).

i- 51324
ii- 23514
iii- 32154

4- if the stack input stream is A B C D E F what is the sequence of operations to get the out put C B D E F A?

5- write a function to check if the stack St is empty.

6- Explain how we can use the record for stack implementation?

7- State the main applications of the stack?

8- Explain how we can use stack to achieve the processing of procedures calls in computer programming?

9- Consider the following program structure

```
Begin   (main program)
....
....
....
Call A
....
....
Call B
....
....
.... Call X ....
....        ....
....        ....
....        ....
....        Call M
....        ....
....        ....
....        Call N
....        ....
....        ....
....   Call Y
....   ....
....   ....
Call C
....
....
```

END

Explain in details how we can use the stack in processing of such program.

10 – Convert the following infix expressions into postfix notations using two stacks:

i- a+b^2/4-(c*5/8-f) ^3

ii- m^3 or n-b/2 and (m+n)

iii- a+b+c*(-d/3^4)and not f

iv- a^(b/2)or(x+y/3-w)and (c^2)^3

v- x^n+(m-p*4)^f and (-b)^2/c+6

vi- a+b-c or (x*2^n)-m/p and f^2

vii- a^(b/2)and (x-y/3+w)or(c^3)^2

viii- n^x-(p*4+m)/f or –c/b^2

ix- b-a+c and n^x-(p/m or f^2)

11-Execute the following postfix notation using the stack ab*cde^/+ when a=5, b=6,c=8,d=2,e=2

12- Execute the following infix notation using the stack a/(b*c/2)^4+m if a=10,b=8,c=4,m=20

13- Write an algorithm that reads in a string and print its characters in reverse order (note: the string terminator is a .‹ Which should not be printed as a part of the reversed string).

14- Write a procedure that that reads in a string and print its characters in reverse order (note: the string terminator is a blank Which should not be printed as a part of the reversed string).

15- Write a procedure to add an element to the queue called (LINE) of size 30 characters.

16- Write a procedure to delete an element from the circular queue called (BOB)of size 15 integers.

17- What are the main differences between simple queue and circular queue? Explain that in detail.

18- What are the applications of the queue in the computer?